

MACHINE LEARNING • BAB 5

5.11 IMPLEMENTASI PYTHON DECISION TREE & PRUNING

Modul lengkap: Praktikum 5.1 – 5.6 dengan penjelasan tiap langkah, output asli, dan interpretasi

Disusun oleh

Ahmad Dandi Subhani

202343500126

5.11. IMPLEMENTASI PYTHON

Bagian ini mempraktikkan algoritma *decision tree* dengan tiga skenario: tanpa pruning (tumbuh penuh), pre-pruning, dan post-pruning (*cost complexity pruning*). Dataset yang dipakai adalah **Breast Cancer Wisconsin** dari *sklearn.datasets* (569 pasien, 30 fitur numerik, 2 kelas: *malignant* dan *benign*).

Praktikum 5.1 Persiapan Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, validation_curve
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
```

Penjelasan: *numpy* dan *pandas* untuk olah data, *matplotlib* untuk visualisasi. Dari *sklearn* diimpor *load_breast_cancer* (dataset), *train_test_split* (split data), *DecisionTreeClassifier* (model pohon keputusan), *plot_tree* (visualisasi pohon), serta *accuracy_score* untuk evaluasi.

Praktikum 5.2 Import Dataset

```
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
print("Ukuran data:", X.shape)
print("Distribusi kelas:", np.bincount(y))
```

```
Ukuran data: (569, 30)
Distribusi kelas: [212 357]
```

Penjelasan: dataset dimuat sebagai *DataFrame* (*as_frame=True*) agar nama fiturnya terbaca. *X* berisi 30 fitur, *y* berisi label (0 = *malignant* sebanyak 212, 1 = *benign* sebanyak 357).

Praktikum 5.3 Preprocessing (Split Data)

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42, stratify=y)
```

Penjelasan: data dibagi 80% latih dan 20% uji. *stratify=y* menjaga proporsi kedua kelas tetap sama di data latih dan uji, sedangkan *random_state=42* membuat pembagiannya dapat diulang (*reproducible*).

Praktikum 5.4 Training Model (dengan dan tanpa Pruning)

```
# Model tanpa pruning (tumbuh penuh)
tree_full = DecisionTreeClassifier(random_state=42)
tree_full.fit(X_train, y_train)

# Model dengan pre-pruning
tree_pruned = DecisionTreeClassifier(max_depth=4,
                                     min_samples_leaf=5, random_state=42)
tree_pruned.fit(X_train, y_train)

# Model dengan post-pruning (Cost Complexity Pruning)
path = tree_full.cost_complexity_pruning_path(X_train, y_train)
ccp_alpha_terpilih = path.ccp_alphas[len(path.ccp_alphas)//2] # contoh pemilihan alpha
tree_ccp = DecisionTreeClassifier(ccp_alpha=ccp_alpha_terpilih,
                                  random_state=42)
tree_ccp.fit(X_train, y_train)
```

Penjelasan: tiga model dilatih untuk dibandingkan.

- **Tanpa pruning** (*tree_full*): pohon dibiarkan tumbuh sampai semua daun murni. Berisiko *overfitting* karena menghafal data latih.
- **Pre-pruning** (*tree_pruned*): pertumbuhan dibatasi sejak awal dengan *max_depth=4* (kedalaman maksimal 4) dan *min_samples_leaf=5* (setiap daun minimal 5 sampel).
- **Post-pruning** (*tree_ccp*): pohon ditumbuhkan penuh dulu, lalu cabang-cabang yang lemah dipangkas berdasarkan *cost complexity pruning path*. Nilai *ccp_alpha* diambil dari tengah *pruning path* sebagai contoh (nilai terpilih: 0,005275).

Praktikum 5.5 Evaluasi

```
for nama, model in [("Tanpa Pruning", tree_full),
                    ("Pre-Pruning", tree_pruned), ("Post-Pruning (CCP)", tree_ccp)]:
    acc_train = accuracy_score(y_train, model.predict(X_train))
    acc_test = accuracy_score(y_test, model.predict(X_test))
    print(f"{nama}: Akurasi Train={acc_train:.3f}, "
          f"Akurasi Test={acc_test:.3f}, Gap={acc_train-acc_test:.3f}")
```

Tanpa Pruning: Akurasi Train=1.000, Akurasi Test=0.912, Gap=0.088

Pre-Pruning: Akurasi Train=0.976, Akurasi Test=0.921, Gap=0.055

Post-Pruning (CCP): Akurasi Train=0.978, Akurasi Test=0.930, Gap=0.048

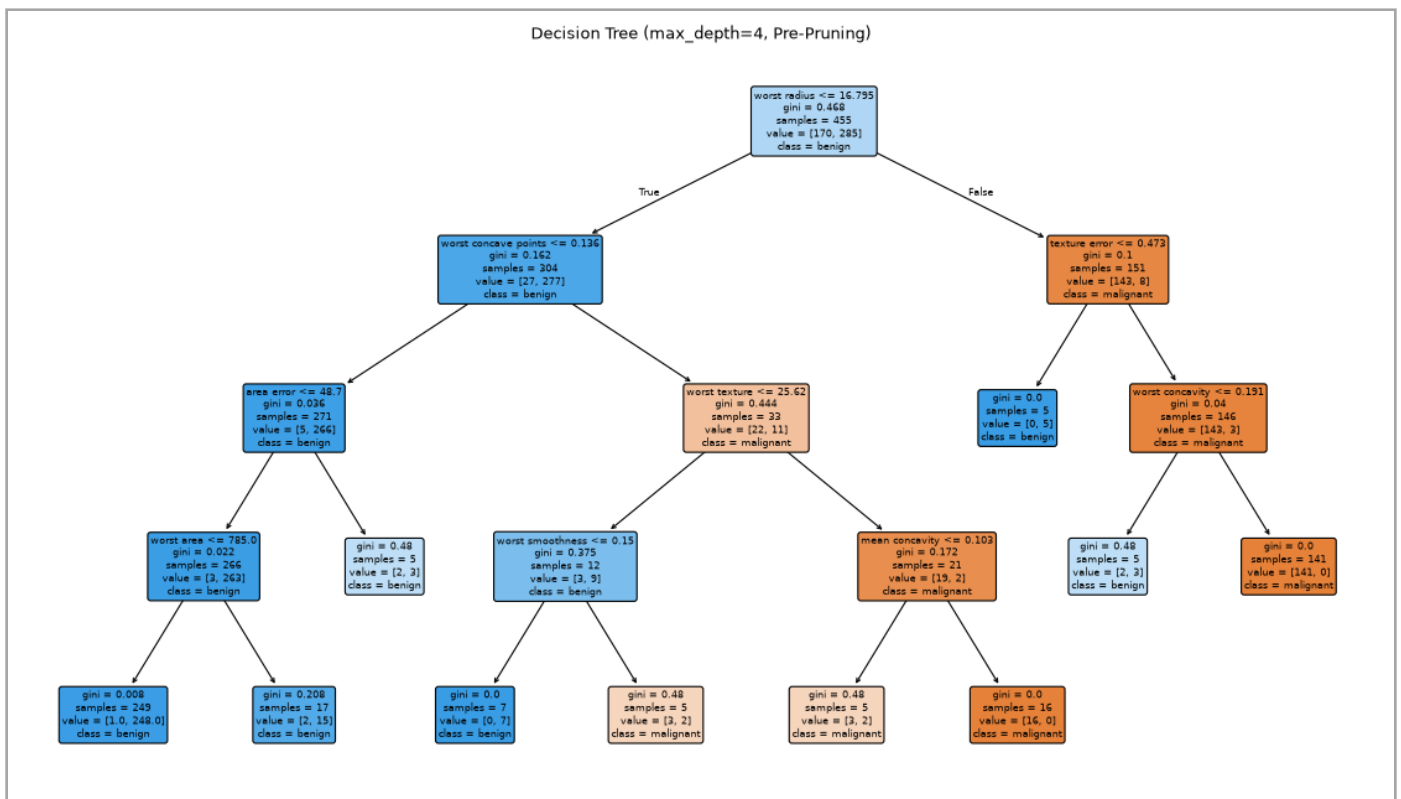
Penjelasan: setiap model diukur akurasi pada data latih dan data uji. *Gap* = selisih keduanya; gap besar menandakan model terlalu menyesuaikan diri dengan data latih (*overfitting*).

Model	Kedalaman	Jumlah daun	Akurasi train	Akurasi test	Gap
Tanpa pruning	7	19	1,000	0,912	0,088
Pre-pruning	4	10	0,976	0,921	0,055
Post-pruning (CCP)	4	7	0,978	0,930	0,048

Praktikum 5.6 Visualisasi

```
plt.figure(figsize=(14,8))
plot_tree(tree_pruned, feature_names=data.feature_names,
          class_names=data.target_names, filled=True, rounded=True,
          fontsize=7)
plt.title("Decision Tree (max_depth=4, Pre-Pruning)")
plt.show()
```

Penjelasan: pohon hasil pre-pruning digambar dengan *plot_tree*. Setiap node menampilkan fitur dan nilai ambang pemisah (misalnya *worst radius* $\leq 16,795$), nilai *gini impurity*, jumlah sampel, dan kelas mayoritas. Warna biru = *benign*, oranye = *malignant*; makin pekat warnanya makin murni node tersebut.



Gambar 5.1: Visualisasi decision tree dengan pre-pruning (max_depth=4, min_samples_leaf=5).

5.11.1. INTERPRETASI

Model tanpa pruning umumnya menunjukkan gap besar antara akurasi train dan test (indikasi *overfitting*), sedangkan model dengan pre-pruning atau post-pruning menunjukkan gap yang lebih kecil, meskipun akurasi train sedikit menurun. Pemilihan strategi pruning terbaik sebaiknya didasarkan pada *validation curve* atau *cross-validation*, bukan sekadar mencoba satu nilai parameter.

TEMUAN DARI HASIL PRAKTIK

- **Tanpa pruning** mencapai akurasi train sempurna (1,000) tetapi gap terbesar (0,088), pola khas *overfitting*: pohon sedalam 7 dengan 19 daun menghafal data latih.
- **Pre-pruning** menurunkan gap menjadi 0,055 dengan akurasi test 0,921, pohon hanya sedalam 4 dengan 10 daun, jauh lebih sederhana dan mudah diinterpretasi.
- **Post-pruning (CCP)** memberi hasil terbaik: akurasi test tertinggi (0,930) dengan gap terkecil (0,048) dan pohon paling ringkas (7 daun).

Kesimpulannya, pruning terbukti mengurangi *overfitting* sekaligus menyederhanakan model. Pada kasus ini post-pruning (CCP) sedikit mengungguli pre-pruning, tetapi pemilihan parameter yang optimal tetap harus melalui *validation curve* atau *cross-validation*, bukan satu nilai coba-coba.

Selesai. Selamat belajar!